Kerbo-S Design Document, Version 2.1 (Sprint 2)

Nov 7, 2026 · @Jaynne

Team: Chance Edwards, Ben Grider, David Pantoja, Jessica Yescas

Kerbo-S is built as a pipeline in which only the rule engine decides what a finding is, and every stage can be tested without the Kerberos lab. Version 2.0 (Oct 29) was the design before any code existed. This version describes the system **as built at the end of Sprint 2** (main branch after pull request #18, 68 automated tests passing) and implements Requirements Document v2.1. Where the code differs from v2.0, Section 2.2 says what changed, why, and what it costs. Parts not yet built keep their v2.0 design and are marked **Sprint 3**.

Status at a glance. Built: data models, the `kadmin` and Kerberos configuration parsers, a fixture-folder loader, eight rules (KC-01 to KC-04, KC-06 to KC-09), the risk scorer, SQLite storage with immutable findings, a command-line audit and a PyQt6 findings window. Sprint 3: the live collector, snapshot export and hash checking, the KDC log and ACL parsers with KC-05, KC-10 and KC-11, the AI Gateway, the Report Builder, and the remaining screens.

1. Design Drivers

Unchanged from v2.0, and each one was tested in Sprint 2:

- **Determinism** (FR-AUD-02, NFR-AUD-01): identical input data and thresholds must give identical findings. Tested by TC-KC-12 (the clean realm audited twice gives identical output).
- **Safety** (NFR-SEC-01, FR-AI-02): collection is read-only, and the AI is isolated from every decision. Sprint 2 code runs no Kerberos commands at all, and the audit completes with the network disabled (TC-KC-13).
- **Testability** (FR-COL-08, NFR-MNT-01): all logic runs from stored fixtures. Every Sprint 2 test runs from files committed to the repository, so no one needs the lab VM to run them.
- **Deliverability:** four developers and limited time favor small, well-known tools over clever ones. Sprint 2 added only one library beyond the standard library and PyQt6: PyYAML.

2. Design Evolution

2.1 From v1.0 to v2.0 (Sprint 1, unchanged)

ID	v1.0 design	v2.0 design	Reason	Trade-off
D-01	Layers connected by direct calls	Pipeline with typed hand-offs (Snapshot, Finding, AiOutput) and a persisted result at each stage	Each stage becomes testable alone and reproducible	More up-front plumbing and data classes
D-02	Collection read the live KDC during every audit	Collectors write hashed raw artifacts and a normalized Snapshot; the rule engine reads only the Snapshot	Develop and test without the VM; reproducible audits; tamper evidence	Two code paths (live and import) and extra disk use
D-03	AI layer after scoring, no defined boundary	AI Gateway with sanitizer, pseudonymizer, versioned prompts, provider interface (OpenAI, mock) and schema validation	Addresses prompt injection and data exposure; AI can be tested offline	Extra component; free-form AI answers are not allowed
D-04	Fixed thresholds and severities in rule text	Thresholds and risk-factor baselines in validated config; severity derived from score	Fixes the KC-02 contradiction; allows tuning in the lab	A bad config could skew results, so it is validated on load
D-05	Single GUI process, Linux and Windows	Headless core library with GUI and CLI front ends; audit runs on a worker thread; Linux only	Responsive UI; the KDC VM has no desktop; same code under test	Thread handling adds complexity

ID	v1.0 design	v2.0 design	Reason	Trade-off
D-06	Report generation inside the AI layer	Separate Report Builder: tables come from the database, AI supplies text only; digest embedded	A report must work with the AI down and be verifiable	Two sources feed each report
D-07	Generic SQLite store	Defined schema; findings immutable through database triggers	Meets FR-DB-02 and TM-05	Triggers must be tested; a local admin can still replace the file

2.2 From v2.0 to v2.1 (Sprint 2, as built)

ID	v2.0 design	As built in Sprint 2	Reason	Trade-off	PR
D-08	One `Snapshot` with principals and artifact hashes	Frozen dataclasses `Principal`, `Policy`, `ConfigSetting` and `Finding`; `Snapshot` holds `principals`, `policies`, `config` and an `errors` list	KC-03 needs policies, KC-07 to KC-09 need settings, and parse errors must travel with the data instead of stopping the audit	`AdminEntry` and `AuthEvent` are not built yet, and the Snapshot does not carry artifact hashes (they are taken when the audit is saved)	#6
D-09	One parser module per source	A `kerbo_s/parsers/` package: `kadmin.py` (principals, policies, classification, dates, durations, folder loader) and `kerberos_conf.py` (MIT profile format for `krb5.conf` and `kdc.conf`)	Keeps the KDC's text formats out of the rules, so rules never read raw text	Parsers depend on the exact `getprinc` labels; a new MIT version may need parser changes	#7, #9, #15
D-10	Collectors produce a snapshot folder	Captured lab output is read from a **fixture folder**: one file per `kadmin` command, the two configuration files, and `collected_at.txt` (the capture time)	Lets the team work with real lab output before the live collector exists; the same folders are the test fixtures	No hashes are checked on load yet; the folder format must stay in step with the future collector	#7, #12, #15
D-11	Settings identified loosely	Each configuration setting has the object ID `file/section/key`, for example `kdc.conf/realms/KERBO.LAB/max_life`	A stable ID lets test manifests match findings exactly on rule ID plus object ID	Long IDs in the table	#6, #9

ID	v2.0 design	As built in Sprint 2	Reason	Trade-off	PR
D-12	KC-06 reads `kadm5.acl` and a justification file	KC-06 checks principals classified as admin (the `/admin` instance) against a justification file whose path is set in `config/rules.yaml`. A missing file means no one is approved, so every admin is reported	Justifications are site data, not code. Failing safe means a lost file cannot hide an unjustified admin	Privileged ACL entries without `/admin` are not checked until the ACL parser exists (Sprint 3)	#9, #10
D-13	KC-04 measured never-used accounts from the creation date	Measured from the last password change date	`getprinc` prints no creation date; the first password is set when the account is created	A reset password on a never-used account restarts the count	#13
D-14	MIT profile files assumed to contain only sections and pairs	The parser skips `include`, `includedir` and `module` lines; the files they name are not read	The real lab `krb5.conf` has `includedir /etc/krb5.conf.d/`; without this the clean realm reported a parse error (defect DF-02)	Settings placed in an included file are not seen yet	#13
D-15	Weak key types matched by name	KC-07 splits each key label on colons and checks every part, so `DEPRECATED:arcfour-hmac` is caught	Newer MIT versions add a `DEPRECATED:` prefix; the planted RC4 key was missed until this fix (defect DF-01)	None found	#13
D-16	Findings sorted by rule and object	Engine output is sorted by score (highest first), then rule ID, then object ID	The dashboard must list findings by score (FR-RSK-04) and the order must still be stable	None found	Starter (Sprint 1)
D-17	Generic error handling	A `SnapshotError` for a missing file, a folder, invalid JSON, missing fields or a bad principal entry. The CLI prints one clear line and exits with **code 2**; the window shows a message box	Users get a clear message, never a traceback (TC-UI-06)	Exit codes become part of the CLI contract and must stay stable	#13
D-18	Schema with seven tables	Five tables built: `audits`, `artifacts`, `findings`, `reviews`, `collection_errors`, with the two immutability triggers. `ai_outputs` and `reports` wait for Sprint 3	Build only the tables that have data in Sprint 2	Sprint 3 adds two tables to an existing schema; `CREATE TABLE IF NOT EXISTS` makes that safe	#16
D-19	Every audit saved	The CLI saves every audit to SQLite (`--db`, default `paths.database` in `config/app.yaml`, which is `audit_output/kerbo_s.db`). A failed save prints an error and exits with **code 3**, after the JSON results are already printed	The results are not lost if the database is unavailable, and a script can tell the two failures apart	The window does not save audits yet	#16

ID	v2.0 design	As built in Sprint 2	Reason	Trade-off	PR
D-20	Lab on VirtualBox	Lab KDC on Proxmox on a team member's home server (see Section 4)	Chosen by the member who built it, for the reasons in Change Document IC-04	Only one member can reach the lab; fixtures in the repository remove the dependency for testing	#10

3. System Architecture

The core library contains no GUI code, so the PyQt6 window, the command-line tool and the test suite all run the same logic. The flow as built in Sprint 2 is:

```
Lab KDC (Proxmox)                          Repository / workstation
-----------------                          ------------------------------------
-------------------------
tools/capture_fixtures.sh  --copy-->  fixture folder  --load_fixture_dir-->
Snapshot
 (kadmin.local, krb5.conf,             (or a JSON snapshot  --load_snapshot--
>  Snapshot)
  kdc.conf, collected_at)
|
                                                                  run_audit:
rules KC-01..KC-09 (8)

|
                                                                  score_finding:
weighted formula

|
                                                          CLI: JSON + save_audit
(SQLite)   Window: table
```

The three trust zones of v2.0 are unchanged: the KDC host, the workstation, and the external AI service. In Sprint 2 nothing crosses into the AI zone.

Component	Responsibility	Module as built	Status	Requirements
GUI (PyQt6)	Open a snapshot or fixture folder, show the findings table with filters and counts, show errors	`ui/main_window.py`, `ui/findings_view.py`	Partial: dashboard built; detail, reports, settings and worker thread Sprint 3	FR-UI-01, 02
CLI	Headless audit of a snapshot; prints JSON; saves to SQLite	`cli.py` (`python -m kerbo_s.cli audit --snapshot FILE`)	Built for audit; collect and export Sprint 3	FR-UI-05
AuditService	Orchestrates stages	`engine.run_audit` (rules and scoring only)	Partial	FR-AUD-01
Collectors	Run allow-listed commands; hash and store raw artifacts	Not built; `tools/capture_fixtures.sh` captures lab output by hand	Sprint 3	FR-COL-01 to 07, 09
Parsers	Turn raw text into typed models	`parsers/kadmin.py`, `parsers/kerberos_conf.py`	Built for principals, policies and both config files; KDC log and ACL Sprint 3	FR-COL-01 to 03, NFR-SEC-04
Snapshot I/O	Load snapshots and folders; export; verify hashes	`snapshot/io.py`, `parsers/kadmin.load_fixture_dir`	Partial: import built; export and hash check Sprint 3	FR-COL-08
Rule Engine	Independent rule classes over a Snapshot	`rules/kc01.py` to `kc09.py`, `rules/lifetimes.py`	8 of 11 rules	FR-AUD-01 to 07
Risk Scorer	Weighted formula, severity bands, factors stored	`scoring/scorer.py`	Built	FR-RSK-01 to 03

Component	Responsibility	Module as built	Status	Requirements
Findings Store	SQLite repository; findings immutable	`store/schema.sql`, `store/repository.py`	Built (five tables)	FR-DB-01, 02
Config Loader	Loads and validates thresholds and factor baselines	`config.py`	Built	FR-AUD-03, FR-RSK-03
AI Gateway	Sanitize, prompt, call provider, validate, fall back	Not built (`config/app.yaml` selects `provider: mock`)	Sprint 3	FR-AI-01 to 09
Report Builder	Technical and executive reports, PDF, digest	Not built	Sprint 3	FR-RPT-01 to 05

4. Deployment View

As built in Sprint 2. The lab KDC is a Proxmox VM (`kerbo-kdc` : 2 CPU, 4 GB RAM, 32 GB disk) running Ubuntu Server 26.04 LTS on a team member's home server, with realm KERBO.LAB and host kdc.kerbo.lab. The full setup is recorded in `docs/lab-setup.md` . A Proxmox snapshot named `clean-baseline` lets the realm be reset before the planted scenarios.

Step	Where	What happens
Capture	KDC VM, with sudo	`tools/capture_fixtures.sh` runs `kadmin.local` list and get commands, copies `krb5.conf` and `kdc.conf`, and writes `collected_at.txt`
Commit	Development VM	The folder is committed under `tests/fixtures/real/` with its `expected_findings.yaml`, written before the audit runs
Audit	Any team machine	pytest, the CLI or the window reads the folder; no access to the lab is needed

Planned (Sprint 3), unchanged from v2.0: headless collection on the KDC with the live collector and snapshot export, copied with scp to an Ubuntu desktop admin VM for analysis, AI and PDF export. The KDC host never needs internet access or an AI key.

Trade-off. The capture is manual and only one member can reach the lab, but the captured folders are real KDC output, they are versioned in Git, and they double as the test fixtures and the evidence trail.

5. Technology Choices

Concern	Choice	Sprint 2 result	Rejected alternative
Language	Python 3.10 or later	Final test run on Python 3.14.4	None
GUI	PyQt6	Findings window built and checked on a Windows laptop	Tkinter (weak tables); web app (out of scope)
Storage	SQLite via `sqlite3`	Built; triggers enforce immutability	JSON files (no queries); PostgreSQL (overkill)

Concern	Choice	Sprint 2 result	Rejected alternative
Configuration	YAML (PyYAML `safe_load`), validated on load	`config/rules.yaml` holds every threshold and factor; a factor outside 0 to 100 is rejected	Hard-coded constants (breaks FR-AUD-03)
Snapshot format	JSON plus raw text files	Both exist: a JSON snapshot (principals only) and the raw fixture folder	Pickle (unsafe to load)
AI	OpenAI Python SDK behind a Provider interface	Not built; model still TBD	Local Llama as primary (kept as R2)
PDF	QTextDocument with QPdfWriter	Not built	ReportLab (fallback)
Testing	pytest with fixtures captured from the lab	68 tests, all from committed files	unittest
Source control	Git and GitHub with pull-request review	18 pull requests in Sprints 1 and 2; a few documentation commits went straight to main (Change Document PC-01)	None

6. Data Design

6.1 Models as built

Model	Fields	Notes
Principal	name, kind (user, service, admin, internal), last password change, last successful authentication, policy, key encryption types	`password_age_days(now)` measures age from the snapshot time, never the clock. Maximum ticket life and attributes are not stored yet

Model	Fields	Notes
Policy	name, minimum length, maximum and minimum life (days), history, maximum failures	Used by KC-03; the rest is ready for later rules
ConfigSetting	file, section, key, value; `object_id` = `file/section/key`	Nested realm blocks give sections such as `realms/KERBO.LAB`
Snapshot	collected_at, principals, policies, config, errors	`errors` holds (source, message) pairs
Finding	rule ID, object type, object ID, observed, threshold, score, severity, factors	Frozen; the scorer returns a new scored copy
AdminEntry, AuthEvent	Not built	Sprint 3, with the ACL and KDC log parsers

Principal classification (FR-AUD-05): `K/M` and names starting `krbtgt/`, `kadmin/` or `kiprop/` are internal; a name ending `/admin` is admin; any other name with an instance is a service; the rest are users.

6.2 SQLite schema as built

Table	Main columns	Notes
audits	id, started_at, mode, tool_version, ruleset_version, config_hash	One row per audit; `config_hash` is the SHA-256 of `rules.yaml`
artifacts	id, audit_id, source, command_or_path, collected_at, sha256, stored_path	One row per input file; the hash must be 64 characters
findings	id, audit_id, rule_id, object_type, object_id, observed, threshold, impact, exploitability, exposure, compliance, score, severity, artifact_id	Immutable: triggers `findings_no_update` and `findings_no_delete` abort with "findings are immutable"

Table	Main columns	Notes
reviews	finding_id, status (default open), note, reviewed_at	The only finding data that can change
collection_errors	id, audit_id, source, message, occurred_at	One row per parse or read error
ai_outputs, reports	As in v2.0	Sprint 3

Each audit is saved in one transaction. Every finding points to the input file it came from: `getprinc_<name>.txt` for a principal, `getpol_<name>.txt` for a policy, and the configuration file for a setting. When the input is a single JSON snapshot, that file is the evidence for every finding.

6.3 Ground-truth manifests

Each fixture folder has an `expected_findings.yaml` with an `expected` list and a `not_expected` list (the boundary cases, such as a password exactly 90 days old). A finding matches when its rule ID and object ID match. The manifests were written before the audit ran. The clean folder expects zero findings; planted expects 8; aged expects 11 (planted plus three age findings from six edited dates, listed in the folder's README).

7. Component Design

7.1 Parsers and the fixture loader

- `parse_principal` and `parse_policy` read `getprinc` and `getpol` output label by label. Dates such as `Fri Aug 07 17:06:59 UTC 2026` and `[never]` and durations such as `1 day 00:00:00` are converted.
- `parse_profile` reads the MIT profile format used by both configuration files, including nested `{ }` blocks and comment lines, and `duration_seconds` handles `10h 0m 0s`, `7d`, `10:00:00` and plain seconds.
- A line that does not match is added to the errors list with its file and line, and parsing continues (FR-COL-07, NFR-SEC-04).
- `load_fixture_dir` reads `collected_at.txt`, `listprincs.txt`, one `getprinc_` file per principal, `getpols.txt` with one `getpol_` file per policy, and `krb5.conf` and `kdc.conf`. A missing file is recorded as an error and the rest still loads.

- **Not built:** the allow-listed collector with timeouts and output caps, hashing before parsing, and the KDC log and ACL parsers.

7.2 Rule engine

Each rule is a class with an `id` and an `evaluate(snapshot, config)` method that yields findings, with no input or output of its own. The engine runs the rules in ID order, scores each finding, and sorts the result by score, then rule, then object.

Rule	Checks (thresholds from `config/rules.yaml`)	Object	Notes
KC-01	User password age over 90 days	principal	Users only
KC-02	Service password age over 180 days	principal	Services only
KC-03	Policy minimum length under 12; user, service or admin principal with no policy	policy or principal	Internal principals skipped
KC-04	User with no successful authentication for over 180 days	principal	Never-used accounts measured from the password change date (D-13)
KC-06	Admin principal not in the justification file	principal	Fails safe if the file is missing (D-12)
KC-07	RC4, arcfour or DES (including 3DES) in `supported_enctypes`, `permitted_enctypes`, `default_tkt_enctypes` or `default_tgs_enctypes`; `allow_weak_crypto = true`; a principal holding such a key	config or principal	Colon-separated labels checked part by part (D-15)
KC-08	`max_life` or `ticket_lifetime` over 24 hours	config	Shared helper in `lifetimes.py`; policy and principal maximum life not yet checked
KC-09	`max_renewable_life` or `renew_lifetime` over 7 days	config	Same helper

Rule	Checks (thresholds from `config/rules.yaml`)	Object	Notes
KC-05, KC-10, KC-11	As in v2.0		Sprint 3, need the KDC log

7.3 Risk scorer

Unchanged from v2.0. `risk_score` applies Impact x 0.40 + Exploitability x 0.30 + Exposure x 0.20 + Compliance x 0.10, rounded to one decimal place, and `severity_for` maps 0 to 24 Low, 25 to 49 Medium, 50 to 74 High and 75 and above Critical. The factor values are copied into the finding and saved with it. All eight Sprint 2 rules score exactly as Requirements Section 6.2 (TC-RSK-01).

7.4 AI gateway and report builder

Not built; the v2.0 design in its Sections 7.4 and 7.5 stands for Sprint 3: select validated findings, sanitize to an allow-list of fields, pseudonymize names, build a versioned prompt, call the provider at temperature 0.2 or lower with a JSON schema, validate and retry once, then store with provenance. Reports take their tables from the database and only narrative text from the AI, with a SHA-256 digest and a verify function.

7.5 User interface as built

Screen	Built in Sprint 2	Sprint 3
Findings window	Buttons to open a JSON snapshot or a fixture folder; severity filter (All, Critical, High, Medium, Low) and category filter (Credentials, Principals, Configuration); a counts line by severity; columns Severity, Score, Rule, Category, Object, Observed, Threshold, Status (New)	Authentication category with KC-10 and KC-11

Screen	Built in Sprint 2	Sprint 3
Error display	A message box for a snapshot that cannot be loaded; a warning listing up to ten unreadable inputs from a folder	
Run Audit progress, Finding Detail, Reports, Settings		All, with the worker thread

The filtering and display logic sits in `findings_view.py` with no Qt code, so it is unit-tested without a display (TC-UI-02, five tests).

7.6 Errors and exit codes

Situation	CLI	Window
Audit completed and saved	JSON on standard output, exit code 0	Table filled
Snapshot missing, a folder, not JSON, missing `collected_at` or a bad principal entry	One-line message, exit code 2	Message box
Results printed but the database save failed	Error message, exit code 3	Not applicable (the window does not save yet)
Unreadable file inside a fixture folder	Recorded in `collection_errors`	Warning box; the rest is shown

7.7 Concurrency

Not built. Audits of the lab-sized fixtures finish in well under a second (68 tests in 0.14 seconds), so the window runs them on the UI thread for now. The QThread worker from v2.0 is planned for Sprint 3 together with live collection, which is the slow step.

8. Key Flows

Audit run as built

1. The user runs the CLI on a JSON snapshot, or opens a snapshot or fixture folder in the window.
2. `load_snapshot` or `load_fixture_dir` builds the Snapshot and collects any read or parse errors.
3. `run_audit` runs the eight rules and scores each finding.
4. The CLI prints the findings as JSON and saves the audit, its input hashes, findings and errors to SQLite; the window shows the table.

The AI analysis and reporting flow is unchanged from v2.0 and is Sprint 3 work.

9. Security Design

Control	Implemented in	Threat	Status
Allow-listed, read-only commands; no shell	Collectors	TM-01, TM-07	Sprint 3. Kerbo-S runs no commands yet
Raw artifact hashing; hash check on import	Store, Snapshot I/O	TM-01, TM-05	Partial: SHA-256 of every input file saved with each audit; check on import Sprint 3
Field allow-list, pseudonymization, fixed prompts, schema validation	AI Gateway	TM-02, TM-03	Sprint 3
API key from an environment variable; never in config or the repository	Config	TM-04	In place: `config/app.yaml` has no key; `.gitignore` excludes key files and `*.keytab`
Owner-only permissions on database, snapshots and reports	Store, Report Builder	TM-06	Sprint 3
Immutable findings	Store	TM-05	Built, TC-DB-01 Pass
Report digest with verify function	Report Builder	TM-05	Sprint 3

Control	Implemented in	Threat	Status
YAML loaded with `safe_load`; snapshots are JSON, never pickle	Config, Snapshot I/O	TM-01	Built
KDC host never calls the AI service	Deployment	TM-03	In place

Team practice. The repository is public, so credentials must never be committed. In Sprint 2 the team also agreed that lab passwords are shared only through Microsoft Teams, never in shared documents, Trello or the repository (Change Document PC-08).

10. Design Strengths, Weaknesses and Trade-offs

10.1 What worked

- **Real lab output as the test fixtures.** Both Sprint 2 defects (DF-01 and DF-02 in the test plan) were invisible with hand-written samples and appeared at once on real `getprinc` and `krb5.conf` output. Committing the captures also means anyone can run the full suite without the lab.
- **Manifests written before the audit.** The planted (8 of 8) and aged (11 of 11) results are checked against ground truth that existed first, including the boundary cases that must not be reported, which gives the detection metrics their meaning.
- **Pure rules and a pure scorer.** No rule reads a file, the network or the clock, so every rule has small boundary tests (91 and 90 days, 181 and 180, 25 and 24 hours, 8 and 7 days).
- **Immutability in the database itself.** The triggers stop changes from any program, not only from Kerbo-S code.
- **UI logic split from Qt.** The filters and counts are tested without a display.

10.2 Weaknesses and remaining issues

Issue	Effect	Planned response
No live collector or snapshot export	Audits read captured output only	Sprint 3 collector with the allow-list, timeouts and hashing

Issue	Effect	Planned response
Hashes are taken when an audit is saved, not at capture, and never checked on import	A changed fixture would not be noticed (TV-14)	Hash at capture; verify on import (TC-COL-09)
JSON snapshot format holds principals only	Policy and configuration rules cannot run from a JSON snapshot	Snapshot export writes all models in Sprint 3
Window does not save audits and runs on the UI thread	Only CLI audits reach the database; a slow live audit would freeze the window	Shared audit service with saving, plus the worker thread
`include` and `includedir` files are not read	A weak setting in an included file would be missed	Follow includes in the collector, with a depth limit
KC-06 covers `/admin` principals only; KC-08 checks configuration, not policy or principal maximum life	Some privileged or long-lived principals are not checked	ACL parser (S2-07); extend KC-08
Lab reachable by one member	Only that member can capture new data	Captures stay in the repository; VirtualBox fallback from `docs/lab-setup.md`
`requirements.txt` uses minimum versions	Installs may differ between machines	Pin versions before TC-PORT-01
No automated test run on pull requests	Tests run only when someone runs them	Add a GitHub Actions workflow

10.3 Decisions carried from v2.0

Decision	Advantages	Disadvantages and risks	Mitigation
Deterministic engine separate from AI	Reproducible, auditable findings; AI failures cannot change results	AI cannot discover issues the rules miss	Add rules as new weaknesses are found; AI correlation adds context

Decision	Advantages	Disadvantages and risks	Mitigation
Snapshot boundary between collection and analysis	Test without the VM; evidence trail; split deployment	Manual copy step; stale snapshots	Show collection time on every screen and report
Parse `kadmin.local` text output	Works on any MIT Kerberos install; no extra libraries	Output format may vary between versions (seen in DF-01)	Fixtures from the lab; strict patterns; errors logged
Rules as Python classes with YAML thresholds	Easy unit tests; thresholds tunable	A new rule needs code	Small base class; eight rules already follow the template
OpenAI as primary AI behind an interface	Best expected quality for the effort	Data leaves the lab; cost; internet dependency	Sanitize and pseudonymize; mock mode; response cache
SQLite with triggers	Simple; no server; tamper-resistant at the row level	A local admin can replace the file	Digest verification; owner-only permissions (Sprint 3)
Linux only	Matches the lab; fewer test cases	Narrower audience than v1.0	Windows clients still audited through KDC logs

11. Requirements Traceability

Requirement group	Design elements	Sprint 2 tests
FR-COL (collection)	Parsers, fixture loader, Snapshot I/O, artifacts table	TC-COL-01, 02, 03, 07; TC-ENV-02
FR-AUD (audit engine)	Rule classes, Config, findings table	TC-KC-01 to 04, 06 to 09, 12, 13, 15
FR-RSK (risk scoring)	Risk Scorer, config baselines	TC-RSK-01, 02

Requirement group	Design elements	Sprint 2 tests
FR-UI and FR-DB	Findings window, CLI, Findings Store	TC-UI-02, 05, 06; TC-DB-01, 02
FR-AI, FR-RPT	AI Gateway, Report Builder	Sprint 3
NFR-SEC and TM-01 to 07	Section 9 controls	TC-DB-01; others Sprint 3

12. Repository Layout as Built

```
Cybersecurity-Capstone/
  README.md  requirements.txt  pytest.ini
  config/            rules.yaml, app.yaml, admin_justifications.example.yaml
  tools/             capture_fixtures.sh
  src/kerbo_s/
    models.py  config.py  engine.py  cli.py
    parsers/         kadmin.py, kerberos_conf.py
    snapshot/        io.py
    rules/           base.py, kc01.py to kc04.py, kc06.py to kc09.py,
lifetimes.py
    scoring/         scorer.py
    store/           schema.sql, repository.py
    ui/              main_window.py, findings_view.py
  tests/
    fixtures/        sample_snapshot.json; real/clean, real/planted,
real/aged; admin_justifications.yaml
    test_parsers/  test_rules/  test_scoring/  test_store/
    test_engine.py  test_end_to_end.py  test_snapshot_errors.py
test_ui_findings_view.py
  docs/              lab-setup.md, evidence/, sprint1/, test runs
```

Not yet present from the v2.0 layout: `collectors/`, `parsers/kdc_log.py`, `ai/`, `reports/` and `rules/kc05.py`, `kc10.py`, `kc11.py`. The repository root also still holds `index.html` and `features.html` from September, which are not part of Kerbo-S.

13. Open Questions

1. Final OpenAI model and its cost per audit (Sprint 3 pilot).

2. KDC log path and message formats on the lab's MIT Kerberos version, for KC-05, KC-10 and KC-11.
3. KC-11 default threshold, after observing normal lab traffic.
4. Whether the GUI installs on an Ubuntu desktop VM without extra system packages.

Closed in Sprint 2: the exact `kadmin.local` labels are confirmed from the captured fixtures (former question 2); KC-04 uses the password change date for never-used accounts (former question 3, D-13).